

Grapple

Game Design Document

Authored by: Leon Kaufmann

Date: April 25th, 2026

Version: 1.0.0

1 Overview

1.1 Game summary

This game is a vertical 2D platformer, and the goal of the player is to get to the top. The player controls a single character and has the ability to perform simple actions like moving and jumping. The way up is designed to present the player with various challenges that he has to overcome. The main tool of the player is a grappling hook that he can use to reach higher platforms. As soon as the player reaches the top, the level is complete, and a new map can be selected.

Each time the player finishes a map, the completion time is saved and compared with previous attempts. This way the player can build up a high score for different levels and try to beat it in the future.

1.2 Inspiration

The main inspiration for this game is the game Sekiro. Sekiro is a soulslike game with one specific feature that inspired me. The grappling hook. In the game, you only have fixed points where you can use the grappling hook and are not free to choose where to attach the hook. This game is inspired by the idea of using the grappling hook freely.



Figure 1: Example from the game Sekiro where the player locks on his grappling hook

The idea for the general design of the game comes from a game called Jump King. The game's goal is to reach the top of a very difficult jump and run, while the player only has the option to charge a jump to advance. If the player misses the targeted platform, the progress of the level is potentially set back quite a bit.

Similar concepts will apply in Climb the Night. The player has to advance in a vertical jump and run with a potential set back if the misses a grappling hook or fails to jump to a targeted platform.



Figure 2: Screenshot from the game Jump King

1.3 Player Experience

The gameplay experience is constantly shifting between two states. The player is standing still and is observing the environment. This is a relaxing state where the player is looking around to find a way to progress. Once he commits to a path he wants to take, the gameplay can switch to a quick, stressful chain of events he has to make, including quick, spontaneous decisions in order for him to advance. A setback at this stage can be very frustrating especially after stressful situations but this is balanced with the reward of completing a stressful section and reaching a checkpoint.

Checkpoints should give the player a big stressrelease because now for the next few sections he has to be less careful about losing much of his progress.

1.4 Plattform

The platform for which the game is developed and aimed for is Linux. Windows compatibility could be achieved but is not planned for at this point.

1.5 Developmentsoftware

For developing the game, a custom engine is used. It is built in C++ and includes various libraries that support building the engine. For rendering, the Simple DirectMedia Layer (SDL2) is used together with the OpenGL Extension Wrangler Library (GLEW). Debug features are added using the Dear ImGui library, FMOD Core (Low-Level) API is used for Audio and Box2D is used for physics simulation.

Because Visual Studio is not available for Linux, the Clion IDE from JetBrains is used to develop the engine and the game.

1.6 Genre

This game is a 2D platformer. It supports speedrunning the game by including features like measured completion time and ghosts of previous attempts.

2 Concept

2.1 Gameplay overview

The player has the ability to use the keyboard to move the character to the left and the right. This alone does not bring the character to other sections of the level and is mainly used for adjusting the player position on a platform. To overcome smaller gaps or obstacles the has a single jump.

The main movement tool is a grappling hook. It can be aimed and fired freely without specific points where the hook has to be locked on to. But there are still areas where the player is not able to grapple onto like spiky walls. After the player shoots the grappling hook, he flies in the aimed direction until the max range of the hook is reached. If the maximum range is reached and the player has not hit a hookable surface, the will not be pulled towards the hook. If he hits it however, the player is moved towards the hook building up some momentum. The player has control over when to release the hook from the surface to fire the hook again.

2.2 Game progression

The progression comes from completing the levels and building up a highscore by completing the levels faster than before.

2.3 Primary Mechanics

The main mechanic is the use of the grappling hook. The entire levels are designed so that the player is forced to use the grappling hook to move through it.

The hook has a limited range and can attach to most surfaces. There are some areas where the hook will not attach to like water, spikes etc.

The player can move the mouse to aim with the grappling hook. While aiming, the player gets a rough indicator of the direction in which the grappling hook will fly. When the hook is released, it tries to attach to the next surface it hits.

1. Hitting a surface

If the player hits a surface, he CAN pull himself towards but does not have to. He also has the option to release the hook at any point after which the player can shoot the hook again.

2. Not hitting a surface

If the player does not hit a surface, he continues falling, or if he stands, nothing happens. After not hitting, he can fire it again.

2.4 Secondary Mechanics

While the player is pulling himself towards the grappling hook, he can build up some momentum. If he is using some smart hooking locations he can build up higher velocities. This mechanic is mainly for speedrunning the levels and not needed to its full extent to complete the levels.

3 Art

3.1 Theme

The game is mainly movement focused and thus a more movement based theme should be used. Although the art does not lean heavily in a theme, it should still convey a ninja like experience.

[you could think about going into the different direction and use a very big person as the character what would not convey a movement based feel but still use it in this movement based game. Mechanics like bouncing could be implemented]

3.2 Design

The background will mostly be white and all elements will be done using simple stick figure sprites and effects. The animations and effects are kept simple. Colors are only used to indicate very important aspects of the game. Everything else is kept in a black and white still figure theme.

4 Audio

4.1 Music

All music in the game is found in the internet and free to use. No self composed music is used.

4.2 Soundeffects

The same thing applies to the sound effects.

5 Game Experience

5.1 UI

The UI will have a very simple structure. Because there are no resources included, the UI only needs to display the passed time in each level.

Outside of the levels there is a single screen to select a level and view the current highscores of the map. After a map is complete, there is a winning screen showing the achieved time and the current highscore of the map. A options menu can be opened to exit a level or the game.

The game contains three interactable UI menus and the game (a single level) itself.

5.2 Controls

The game is playable with mouse and keyboard, as well as with a controller.

Keyboard:

- Waking left/right - A/S
- Jumping - Space
- Aiming - 
- Shooting Hook - 
- Pulling -  (hold)

Controller:

- Waking left/right - 
- Jumping - 
- Aiming - 
- Shooting Hook - 
- Pulling -  (hold)

6 MVP

Simple level selection screen with a single level.

Very simple level design with a start and a end point. The player has a very simple implementation of a grappling hook able to aim and pull himself towards the hook. No momentum implemented yet.

After the player reaches the endpoint he returns to the level select screen.

I Inline Graphics Demo

This template supports a variety of inline graphics, which can be used for a variety of purposes like controller inputs (ie. + to attack up) or for specific dice rolls (ie. ). They can be used by just writing `\controllerA` (replace with whichever inline graphic you want to use) directly within text, no need to fiddle with environments or filepaths or anything like that.

A complete list of all possible inline graphics are included:

I.i Controls

- | | |
|--|---|
| •  <code>\controllerA</code> | •  <code>\controllerJoystickDown</code> |
| •  <code>\controllerB</code> | •  <code>\controllerJoystickRight</code> |
| •  <code>\controllerX</code> | •  <code>\controllerJoystickPress</code> |
| •  <code>\controllerY</code> | •  <code>\controllerRJoystick</code> |
| •  <code>\controllerO</code> | •  <code>\controllerRJoystickUp</code> |
| •  <code>\controllerSquare</code> | •  <code>\controllerRJoystickLeft</code> |
| •  <code>\controllerTriangle</code> | •  <code>\controllerRJoystickDown</code> |
| •  <code>\controllerL</code> | •  <code>\controllerRJoystickRight</code> |
| •  <code>\controllerLalt</code> | •  <code>\controllerRJoystickPress</code> |
| •  <code>\controllerR</code> | •  <code>\controllerLJoystick</code> |
| •  <code>\controllerRalt</code> | •  <code>\controllerLJoystickUp</code> |
| •  <code>\controllerZ</code> | •  <code>\controllerLJoystickLeft</code> |
| •  <code>\controllerPlus</code> | •  <code>\controllerLJoystickDown</code> |
| •  <code>\controllerMinus</code> | •  <code>\controllerLJoystickRight</code> |
| •  <code>\controllerDpad</code> | •  <code>\controllerLJoystickPress</code> |
| •  <code>\controllerDpadUp</code> | •  <code>\controllerMouse</code> |
| •  <code>\controllerDpadLeft</code> | •  <code>\controllerMouseLMB</code> |
| •  <code>\controllerDpadDown</code> | •  <code>\controllerMouseRMB</code> |
| •  <code>\controllerDpadRight</code> | •  <code>\controllerMouseScroll</code> |
| •  <code>\controllerJoystick</code> | •  <code>\controllerMouseScrollUp</code> |
| •  <code>\controllerJoystickUp</code> | •  <code>\controllerMouseScrollDown</code> |
| •  <code>\controllerJoystickLeft</code> | |

I.ii Dice

- | | |
|---|--|
| •  <code>\diceOne</code> | •  <code>\diceFour</code> |
| •  <code>\diceTwo</code> | •  <code>\diceFive</code> |
| •  <code>\diceThree</code> | •  <code>\diceSix</code> |